[INSTITUTION NAME]

Department of Computer Science and Engineering

WildSnake

Snake Game Engine: Object-Oriented Design and Implementation

Mini Project Report

Course: CS3104 – Object-Oriented Programming in C++

Student 1	**Student 2**
Sampath Kumar Reddy	Syed Adam
Roll No. 251158	Roll No. 251156

Academic year and semester:	2nd Year, Semester 3
Submission date:	29 October 2026

Final project report

Declaration

We declare that the project contribution described in this report, titled *WildSnake*, was developed as a CS3104 mini project. The implementation details and limitations are reported to the best of our knowledge. Any external libraries used are identified in the report. The student authors are listed on the cover page.

Sampath Kumar Reddy
Roll No. 251158

Date: 29 October 2026

Syed Adam
Roll No. 251156

Date: 29 October 2026

Acknowledgement

We thank the course instructor for the instruction and feedback provided during this mini project. (Edit this section to acknowledge any additional contributors.)

Abstract

WildSnake is a small snake-game engine implemented in C++17 and presented through a browser interface. The engine models the snake, food, board, score, and game state as separate C++ classes. A lightweight HTTP server exposes state, direction, tick, and reset operations to a JavaScript front end. The project demonstrates encapsulation, composition, an abstract observer interface, standard-library containers, and separation between game rules and presentation. The repository also contains a CMake build and a set of assertion-style tests. The report documents the current source structure and does not claim that the project has been compiled or that the tests have passed.

Keywords: C++, object-oriented programming, game engine, Snake, REST API, UML, CMake

Contents

1 Introduction **1**

1.1 Background . . . 1

1.2 Project overview . . . 1

1.3 Problem statement . . . 1

1.4 Objectives . . . 1

2 Scope and Requirements **3**

2.1 Scope . . . 3

2.2 Functional requirements . . . 3

2.3 Non-functional requirements . . . 4

2.4 Use cases . . . 4

3 Design and Architecture **5**

3.1 Layered architecture . . . 5

3.2 Class design and OOP concepts . . . 5

3.3 UML class specification tables . . . 6

3.4 Sequence: one game tick . . . 7

3.5 Activity: tick and collision handling . . . 8

4 Implementation **9**

4.1 Technology stack and modules . . . 9

4.2 Representative OOP code . . . 9

4.2.1 Encapsulated game model and observer abstraction . . . 9

4.2.2 Direction validation and movement . . . 10

4.2.3 Tick orchestration and composition . . . 11

4.3 HTTP API . . . 11

5 Testing and Results **12**

5.1 Test approach 12
5.2 Test status and observed repository limitations 12
5.3 Screenshots 13

6 Conclusion and Future Work **15**

References **16**

A Build and Run Guide **17**

B Submission Completion Checklist **18**

List of Figures

2.1 Use-case view of player interactions 4

3.1 WildSnake high-level architecture 5
3.2 Core class relationships. ScoreManager is currently standalone. 6
3.3 Sequence for a browser-requested game tick 7
3.4 Activity diagram for a game tick 8

5.1 WildSnake initial screen (replace placeholder with a genuine capture) . . . 13
5.2 Active game state (replace placeholder with a genuine capture) 14

List of Tables

2.1 Functional requirements reflected in the current source 3

3.1 UML class table: model data types and entities 6
3.2 UML class table: game coordination and score utility 7

4.1 Project modules 9
4.2 Backend API operations 11

5.1 Coverage in the existing test source (not a claim of passing tests) 13

Chapter 1

Introduction

1.1 Background

Snake is a grid-based game in which the player steers a growing snake to collect food while avoiding the board boundary and the snake's own body. Its compact rules make it useful for demonstrating state, object collaboration, input validation, collision checks, and separation of responsibilities.

1.2 Project overview

WildSnake separates the rule engine from the graphical interface. The C++ backend owns the authoritative game state. An HTTP API lets the front end read the state and request direction changes, game ticks, and resets. The interface uses HTML, CSS, and JavaScript with a canvas for the board.

1.3 Problem statement

The project aims to provide a maintainable, small-scale implementation of the classic Snake rules, with a clear object-oriented model and a usable interface. The design should make core entities and their responsibilities understandable to a student reviewing OOP concepts.

1.4 Objectives

- Model game entities with focused C++ classes and private state.
- Implement movement, direction validation, food placement, scoring, and collision/game-over behavior.
- Expose game operations through a small JSON HTTP API.

- Provide keyboard, on-screen, start, pause, and restart controls.
- Document the architecture and test the core behavior.

Chapter 2

Scope and Requirements

2.1 Scope

The current implementation has a rectangular, non-wrapping board, a snake starting at the board center, randomly placed food, score increments when food is eaten, and game-over detection for wall, self, and obstacle collisions. The world contains an obstacle collection, although the current constructor does not populate it. The browser UI supports arrow keys and WASD, clickable direction buttons, pause/resume, and restart. The backend listens on `127.0.0.1:8080`.

The score manager can load and save a high score in a text file, but it is not connected to the `Game` score lifecycle. The browser's high-score presentation therefore should not be described as integrated persistent scoring without further implementation and verification.

2.2 Functional requirements

Table 2.1: Functional requirements reflected in the current source

ID	Requirement
FR-01	Display current game state, including board dimensions, snake, food, obstacles, score, and game-over flag.
FR-02	Accept one of four cardinal direction requests.
FR-03	Advance the snake one cell per tick and check collisions.
FR-04	Increase the score and place new food when the snake reaches food.
FR-05	Reset the snake and score and resume active play.
FR-06	Allow browser controls by keyboard and on-screen buttons.
FR-07	Support pause/resume in the front-end game loop.
FR-08	Load and save a high score through the standalone ScoreManager class.

2.3 Non-functional requirements

The code targets C++17 and uses CMake. The engine should keep game rules independent from browser rendering, validate board dimensions and directions, and provide readable class interfaces. The server uses a mutex around shared game operations. Third-party dependencies declared by CMake are cpp-httplib and nlohmann-json.

2.4 Use cases

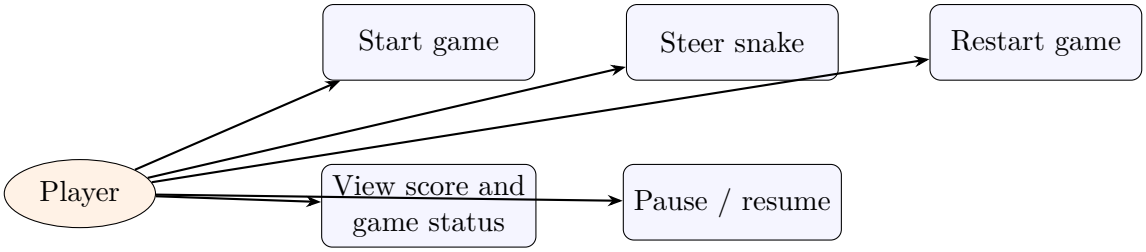


Figure 2.1: Use-case view of player interactions

Chapter 3

Design and Architecture

3.1 Layered architecture

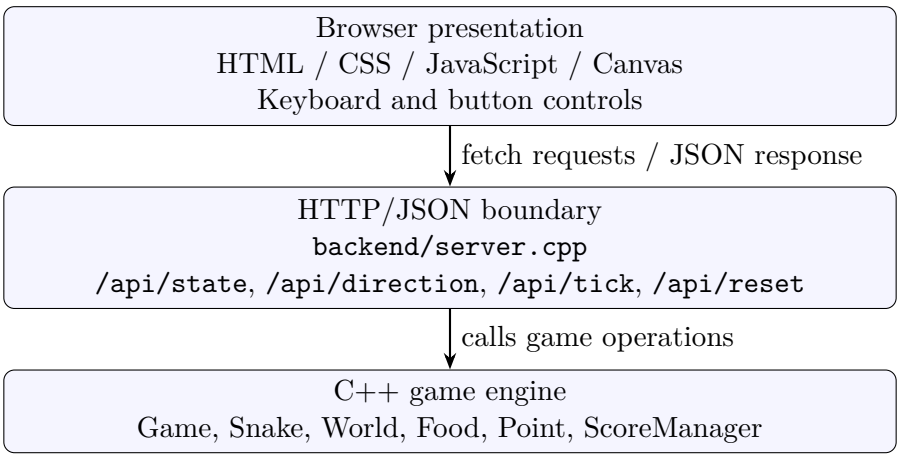


Figure 3.1: WildSnake high-level architecture

3.2 Class design and OOP concepts

- **Encapsulation:** game state is held in private members and read through public methods such as `getScore()` and `getBody()`.
- **Composition:** `Game` owns a `Snake` and a `World`; `World` owns a `Food` value.
- **Abstraction and runtime polymorphism:** `GameObserver` declares virtual event callbacks, and `Game` calls them through the interface. A concrete observer can be supplied by a client.
- **Standard library:** `std::deque<Point>` represents the body, vectors represent obstacles and occupied cells, and `std::mt19937` supports randomized food placement.
- **Value type:** `Point` represents a coordinate and defines equality operators used by collision and collection logic.

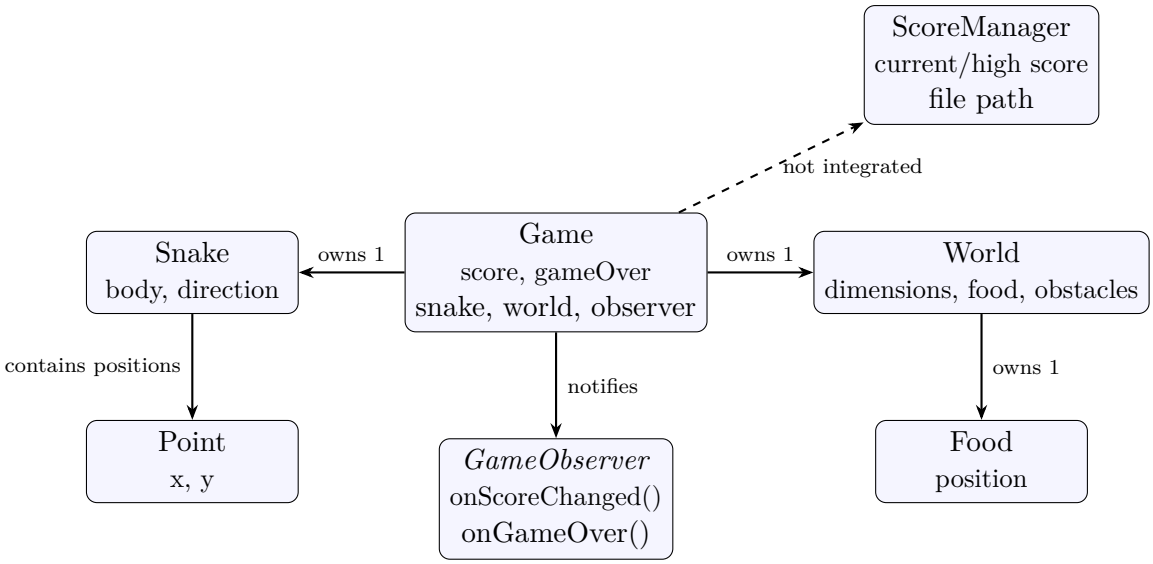


Figure 3.2: Core class relationships. ScoreManager is currently standalone.

3.3 UML class specification tables

The following UML-style tables list the declared attributes and public operations for every class in the game model. A minus sign denotes private visibility and a plus sign denotes public visibility. The `GameObserver` callbacks are virtual; `ScoreManager` remains a separate utility in the current implementation.

Table 3.1: UML class table: model data types and entities

Class	Attributes (visibility)	Public operations (visibility)
`Point`	`x: int`, `y: int` (public value fields)	`Point(x=0,y=0)`; `operator==`; `operator!=`.
`Direction` (enum)	Values: `UP`, `DOWN`, `LEFT`, `RIGHT`	Scoped enumeration used by `Snake` and the API.
`Snake`	`body_: deque<Point>`, `direction_: Direction`, `growing_: bool` (private)	`Snake(start)`; `getBody()`; `getDirection()`; `setDirection()`; `move()`; `grow()`; `occupies()`; `hitsItself()`.
`Food`	`position_: Point` (private)	`Food(position)`; `getPosition()`; `setPosition()`.
`Grid<T>`	`width_: int`, `height_: int`, `cells_: vector<T>` (private)	`Grid(width,height)`; `getWidth()`; `getHeight()`; `contains(Point)`; `at(Point)` (mutable and const overloads). Methods are declared in the header; definitions are absent from the supplied source.

Class	Attributes (visibility)	Public operations (visibility)
`World`	`width_: int`, `height_: int`, `food_: Food`, `obstacles_: vector<Point>`, `rng_: mt19937` (private)	`World(width,height)`; `getWidth()`; `getHeight()`; `getFood()`; `getObstacles()`; `isInside(Point)`; `hasObstacle(Point)`; `placeFood(occupied)`.

Table 3.2: UML class table: game coordination and score utility

Class	Attributes (visibility)	Public operations (visibility)
`GameObserver` (abstract interface)	No data members declared	`onScoreChanged(score)` (public pure virtual); `onGameOver()` (public pure virtual); virtual destructor.
`Game`	`snake_: Snake`, `world_: World`, `score_: int`, `gameOver_: bool`, `observer_: unique_ptr<GameObserver>` (private)	`Game(width,height)`; `setObserver()`; `changeDirection()`; `tick()`; `reset()`; `getScore()`; `isGameOver()`; `getSnake()`; `getWorld()`.
`ScoreManager`	`currentScore_: int`, `highScore_: int`, `filePath_: string` (private)	`ScoreManager(filePath)`; `resetCurrentScore()`; `addPoints(points)`; `getCurrentScore()`; `getHighScore()`; `loadHighScore()`; `saveHighScore() const`. This utility is not currently composed into `Game`.

3.4 Sequence: one game tick

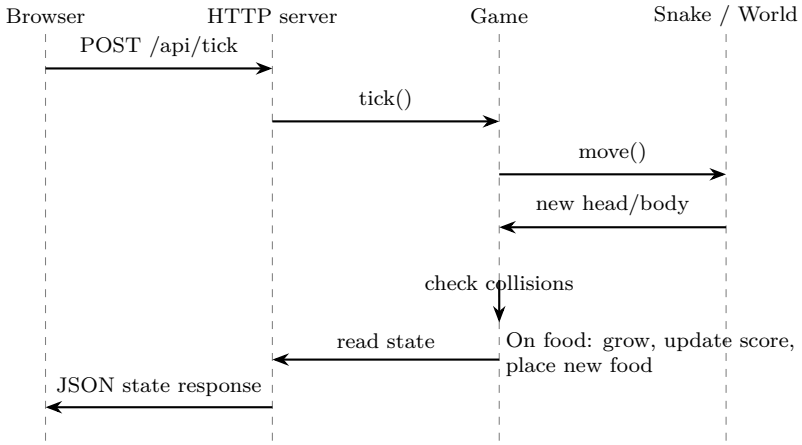


Figure 3.3: Sequence for a browser-requested game tick

3.5 Activity: tick and collision handling

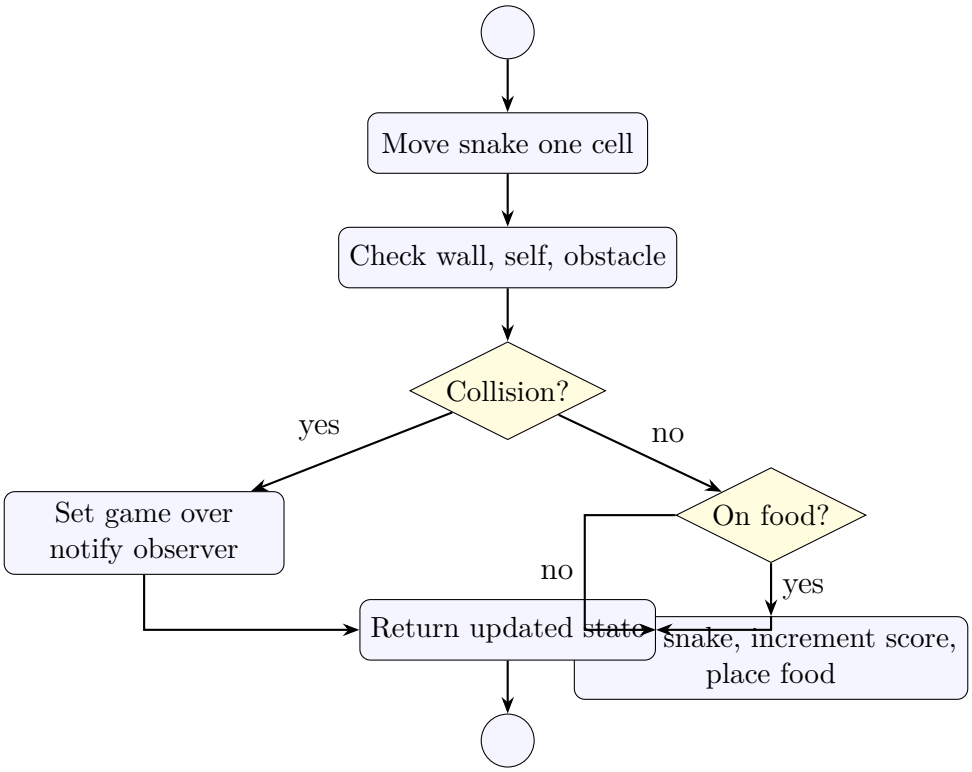


Figure 3.4: Activity diagram for a game tick

Chapter 4

Implementation

4.1 Technology stack and modules

Table 4.1: Project modules

Module	Responsibility
`Snake`	Body, direction validation, movement, growth, self-collision.
`World` and `Food`	Bounds, obstacles, food position, randomized placement.
`Game`	Coordinates ticks, collisions, score, reset, observer notifications.
`ScoreManager`	Reads/writes persistent high score in a text file.
`server.cpp`	Serializes state as JSON and routes HTTP operations.
Frontend	Canvas rendering, controls, API calls, and game-loop presentation.
Tests	Small executable covering points, snake, world, grid, and basic game state.

4.2 Representative OOP code

The following excerpts are drawn from the current source with formatting normalized for the report.

4.2.1 Encapsulated game model and observer abstraction

Listing 4.1: Game owns collaborators and exposes a narrow interface

```
class GameObserver {
public:
    virtual void onScoreChanged(int score) = 0;
```

```
    virtual void onGameOver() = 0;
    virtual ~GameObserver() = default;
};

class Game {
private:
    Snake snake_;
    World world_;
    int score_;
    bool gameOver_;
    std::unique_ptr<GameObserver> observer_;
public:
    void tick();
    void reset();
    int getScore() const;
    bool isGameOver() const;
};
```

4.2.2 Direction validation and movement

Listing 4.2: Snake validates directions and moves its deque body

```
void Snake::setDirection(Direction direction)
{
    switch (direction) {
        case Direction::UP: case Direction::DOWN:
        case Direction::LEFT: case Direction::RIGHT:
            break;
        default:
            throw std::invalid_argument("Invalid snake direction");
    }
    // A multi-segment snake cannot reverse into itself.
    if (body_.size() > 1 && isOpposite(direction_, direction))
        return;
    direction_ = direction;
}

void Snake::move()
{
    Point head = body_.front();
    advance(head, direction_);
    body_.push_front(head);
    if (growing_) growing_ = false;
    else body_.pop_back();
}
```

Note: `isOpposite` and `advance` above are explanatory helpers; the repository implements these checks directly in the method body.

4.2.3 Tick orchestration and composition

Listing 4.3: Game coordinates movement, collisions, and food handling

```cpp
void Game::tick()
{
    if (gameOver_) return;
    snake_.move();
    checkCollisions();
    if (gameOver_) return;

    if (snake_.getBody().front() == world_.getFood().getPosition()) {
        snake_.grow();
        updateScore();
        // Build occupied cells and request the next food position.
        world_.placeFood(occupiedCells());
    }
}
```

Note: `occupiedCells()` abbreviates the vector construction in the source; the repository builds that vector from the snake body.

4.3 HTTP API

Table 4.2: Backend API operations

Method	Path	Behavior
GET	`/api/state`	Returns dimensions, snake, food, obstacles, score, gameOver.
POST	`/api/direction`	Accepts JSON `{"direction":"UP"}`; rejects invalid input.
POST	`/api/tick`	Advances one step and returns updated state.
POST	`/api/reset`	Resets the engine and returns new state.
GET	`/`	Returns server name/status and endpoint names.

Chapter 5

Testing and Results

5.1 Test approach

The repository contains `tests/game_tests.cpp`, a standalone test program registered with CTest when `BUILD_TESTING` is enabled. Its checks cover point equality, initial and changed snake movement, invalid direction rejection, world boundaries and minimum size, grid access, initial game state, one tick, and reset. The test file expresses checks by printing PASS/FAIL and returning a failing exit code if any check fails.

5.2 Test status and observed repository limitations

The report was prepared by source inspection; no build or test execution is claimed. Before submission, compile the project and record the actual command, environment, and result here. At source inspection, there are case/name inconsistencies that may prevent a clean build on case-sensitive systems: `point.h` versus includes of `Point.h`, and `ScoreMananger.h` versus includes of `ScoreManager.h`. Also, `Grid<T>` declares template methods in `World.h`, but their definitions are not present in the supplied source files even though the tests instantiate `Grid<int>`. Resolve these and rerun the tests before submitting.

Table 5.1: Coverage in the existing test source (not a claim of passing tests)

Area	Checks represented
Point	Equality and inequality.
Snake	Initial position, movement, direction change, self-hit baseline, invalid direction.
World and Grid	In-bounds checks, minimum dimensions, element access and out-of-range behavior.
Game	Initial score/state, one tick, score/state after reset.
Additional recommended cases	Eat/grow/score, wall collision, self-collision, reverse-direction prevention, no free food cells, observer callbacks, server request validation.

5.3 Screenshots

No screenshot image files were present in the supplied project workspace. Insert authentic captures of the running interface and, if useful, a terminal showing the backend and test result. Do not submit the following placeholders as evidence; replace them with actual images saved under `report-assets/`.

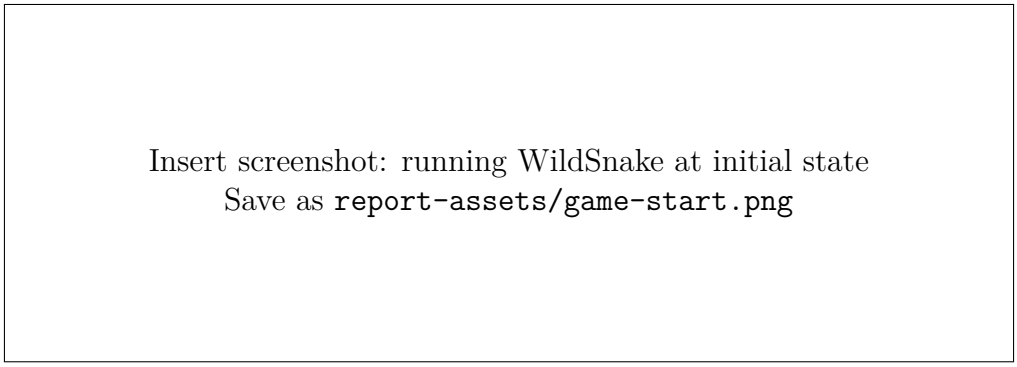


Figure 5.1: WildSnake initial screen (replace placeholder with a genuine capture)

Insert screenshot: active game showing score and snake
Save as `report-assets/game-playing.png`

Figure 5.2: Active game state (replace placeholder with a genuine capture)

Chapter 6

Conclusion and Future Work

WildSnake is a useful OOP mini-project foundation: its engine is divided into small classes, exposes a browser API, and represents core gameplay concepts with private state and explicit interfaces. The TikZ diagrams describe the intended relationships and the current tick flow. The test source covers several basic cases, but the build and test outcome still needs to be verified.

Recommended next steps are to fix include/file-name consistency, implement or remove the currently undefined Grid template, connect persistent score management if high-score behavior is a requirement, and add tests for food, growth, collisions, and observer notifications. Further extensions could add configurable obstacles, difficulty levels, sound, and a packaged release.

References

1. ISO C++ Foundation, “C++ documentation,” https://isocpp.org/.
2. CMake, “CMake Documentation,” https://cmake.org/documentation/.
3. yhirose, “cpp-httplib,” https://github.com/yhirose/cpp-httplib.
4. nlohmann, “JSON for Modern C++,” https://github.com/nlohmann/json.
5. TikZ and PGF, “TikZ and PGF Manual,” https://tikz.dev/.
6. WildSnake project source code, supplied project repository, accessed [enter date].

Appendix A

Build and Run Guide

The CMake configuration declares the C++17 engine, HTTP server, smoke executable, and CTest target. From the project root, a typical build sequence is shown below. Dependency fetching may require network access at configure time. Run these commands after correcting the source naming and Grid issues; record the actual results in Chapter 6.

```
cmake -S backend -B build -DBUILD_TESTING=ON
cmake --build build
ctest --test-dir build --output-on-failure
```

Appendix B

Submission Completion Checklist

- Replace the institution name and repository access date if required by the department's format.
- Add genuine application screenshots in `report-assets/`.
- Fix the noted source/build issues and run the build and tests; report actual outcomes.
- Confirm the instructor's supplied template and apply any required cover-page, certificate, or formatting changes. The template mentioned in the request was not present in the workspace or chat attachment.
- Compile this file with a LaTeX installation that includes TikZ and listings, and inspect the generated PDF for layout and diagram issues.